Análise Exaustiva da Arquitetura do Driver AMDGPU para a APU Cyan Skillfish (AMD BC-250) e Transição para WDDM

Introdução e Contextualização do Hardware Cyan Skillfish

A placa AMD BC-250, baseada na Unidade de Processamento Acelerado (APU) com o nome de código "Cyan Skillfish" (conhecida internamente na sua variante de consola como "Oberon"), representa uma configuração de silício altamente especializada e idiossincrática. Originalmente concebida a partir do APU da PlayStation 5 e posteriormente adaptada para cargas de trabalho de mineração de criptomoedas e processamento de uso geral, esta arquitetura apresenta peculiaridades fundamentais em relação às Unidades de Processamento Gráfico (GPUs) discretas convencionais da AMD[1].

Compreender o comportamento exato do driver amdgpu no kernel Linux para a APU Cyan Skillfish é o alicerce absoluto para a reestruturação e desenvolvimento de um driver base no ecossistema Windows, operando sob o Windows Display Driver Model (WDDM). O objetivo de estruturar uma base WDDM sem APIs gráficas implementadas — um driver bare-metal com acesso total à GPU — alinha-se perfeitamente com o Microsoft Compute Driver Model (MCDM), que permite o controlo total de hardware, submissão de comandos e gestão de memória virtual sem a necessidade imediata de suportar renderização Direct3D completa[4].

A placa integra 6 núcleos de processamento baseados na microarquitetura Zen 2 e uma iGPU de arquitetura RDNA 2 (identificada no ecossistema LLVM e Mesa como GFX1013) com 24 Unidades de Computação (Compute Units - CUs) ativas, partilhando um total de 16 GB de memória GDDR6[1]. O desenvolvimento de um Kernel Mode Driver (KMD) requer o mapeamento meticuloso das sequências de inicialização, gestão de memória, contornamento de mecanismos de segurança de firmware e submissão de comandos que o driver Linux realiza nativamente.

Para efeitos de identificação no barramento Peripheral Component Interconnect (PCI), a variante Cyan Skillfish (e a sua iteração direta, Cyan Skillfish2) é identificada num espectro de IDs de dispositivo, nomeadamente 0x13FE, 0x13F9, 0x13FA, 0x13FB e 0x143F[7]. O driver AMDGPU agrupa estes identificadores sob as flags internas CHIP_CYAN_SKILLFISH e, de forma crítica para a gestão de firmware, AMD_APU_IS_CYAN_SKILLFISH2[10].

Especificação do Hardware	Detalhe Técnico (AMD BC-250 / Cyan Skillfish)

Arquitetura da GPU	RDNA 2 (Identificador Mesa/LLVM: GFX1013)[1]
Unidades de Computação	24 CUs ativas (de um total físico de 36 no die original)[1]
Memória	16 GB GDDR6 Unificada (Barramento de 256-bit, ~448 GB/s)[1]
Engine de Display	DCN 2.01 (Display Core Next modificado e retroativo)[3]
Video Core Next (VCN)	Desativado por firmware (VCN firmware bloqueado)[1]
PCI IDs Principais	0x1002:0x13FE, 0x1002:0x143F [cite: 7, 10]

Evolução Arquitetural: O Contexto RDNA 1 e RDNA 2 no AMDGPU

A arquitetura RDNA (Radeon DNA) marcou uma rutura profunda com a antiga arquitetura GCN (Graphics Core Next), refletindo-se intensamente no design do driver AMDGPU. O RDNA 1 (série Navi 1x) introduziu o Workgroup Processor (WGP) como a unidade fundamental de computação, abandonando a dependência estrita das CUs isoladas, e implementou uma nova hierarquia de cache com a introdução da Cache L1 unificada. O RDNA 2 (série Navi 2x, e a base do Cyan Skillfish) iterou sobre este design com a introdução de aceleradores de Ray Tracing, Infinity Cache (ausente no design específico do Cyan Skillfish devido à sua natureza unificada de memória APU) e suporte para funcionalidades avançadas de API[1].

No contexto do driver AMDGPU, a transição para RDNA 1 e 2 cristalizou o modelo System on Chip v15 (SoC15). Neste paradigma, o hardware deixou de ser tratado como um bloco monolítico com registos indexados de forma global. Em vez disso, a GPU é gerida como um conjunto federado de Blocos de Propriedade Intelectual (IP Blocks), cada um com o seu próprio espaço de endereçamento, mecanismos de reset e gestão de energia. Para dominar a placa BC-250 num projeto WDDM, é necessário compreender que o código RDNA 2 não acede à GPU diretamente, mas roteia todas as interações através de instâncias de IP[16].

O Cyan Skillfish posiciona-se de forma peculiar nesta evolução. Embora possua a capacidade computacional do RDNA 2 (GFX10.3), a sua identificação de versão de IP cai frequentemente num estado híbrido ou "legacy" dentro do driver, partilhando o controlador de memória GMC v10 com o RDNA 1, mas exigindo parametrizações douradas (golden settings) específicas da sua variante GFX10.1.x ou GFX10.3.x, exigindo que a inicialização do SoC15 seja tratada com exceções contínuas no código[10].

A Sequência de Inicialização (Boot Sequence) do Driver AMDGPU

A arquitetura do driver amdgpu para Linux é fortemente modular. O processo de inicialização de um dispositivo AMD começa no nível do subsistema PCI do kernel e flui por uma sequência determinística que o driver WDDM a ser desenvolvido deverá emular estritamente utilizando as interfaces nativas do kernel do Windows (tais como DxgkDdiAddDevice, DxgkDdiStartDevice e os callbacks de inicialização do adaptador)[4].

Da Sonda PCI à Inicialização Base do Dispositivo

No sistema operativo Linux, o ciclo de vida do driver inicia-se de forma assíncrona com a função amdgpu_pci_probe[20]. Esta função é responsável por intercetar a presença do dispositivo PCI correspondente aos IDs registados (neste caso, 0x13FE), ler as suas regiões Base Address Register (BARs) primárias e assegurar que o mapeamento de memória PCIe está funcional[21]. Após validações iniciais, a rotina invoca amdgpu_driver_load_kms para preparar a infraestrutura Kernel Mode Setting (KMS), que atua como o esqueleto para a renderização do framebuffer[21].

Imediatamente a seguir, o fluxo de execução entra na função nevrálgica amdgpu_device_init[25]. É neste exato ponto que o driver averigua o identificador do chip, define o asic_type para CHIP_CYAN_SKILLFISH e aloca a estrutura omnipresente amdgpu_device (frequentemente instanciada como adev em todo o código C). No contexto do desenvolvimento Windows, este processo encontra um paralelo direto e imediato na rotina DriverEntry — onde se preenche a estrutura WDDM DRIVER_INITIALIZATION_DATA com os callbacks implementados — e, subsequentemente, na receção do evento de inicialização de hardware fornecido pelo subsistema de gráficos DirectX (Dxgkrnl) através da chamada a DxgkDdiAddDevice e preparação do contexto do miniporto[4].

Nesta fase preliminar, o amdgpu_device_init lida também com a descoberta dos registos MMIO (Memory-Mapped I/O). A função amdgpu_discovery_reg_base_init é invocada[28]. Curiosamente, enquanto GPUs convencionais dependem de uma tabela de descoberta de IP armazenada na placa para determinar onde os registos de cada bloco IP residem na memória, o Cyan Skillfish contém blocos lógicos rígidos, codificados no driver através da função cyan_skillfish_reg_base_init(adev). Este desvio é um artefacto do seu design original para um ambiente embebido fechado, obrigando o programador do KMD no Windows a mapear as bases de registo (SMC, GFX, MMHUB) através de offsets estáticos em vez de confiar no IP Discovery nativo[28].

O Paradigma sw_init e hw_init: A Orquestração de IP Blocks

A inicialização do silício da GPU propriamente dita reside na iteração governada pela função amdgpu_device_ip_init[29]. A arquitetura da AMD, fiel ao seu modelo SoC15, orquestra a inicialização iterando sobre um array de blocos IP definidos previamente. Para o Cyan Skillfish, a matriz de IPs é configurada principalmente no ficheiro nv.c sob o caso CHIP_CYAN_SKILLFISH e inclui subsistemas essenciais[32]:

- **nv_common_ip_block**: Trata das funcionalidades de nível de sistema do SoC15.

- **gmc_v10_0_ip_block**: Graphics Memory Controller versão 10. Responsável pela gestão da Translation Lookaside Buffer (TLB), Page Tables, GART (Graphics Address Remapping Table) e pela hierarquia de VRAM[10].
- **gfx_v10_0_ip_block**: O próprio motor gráfico e de computação (RDNA 2) onde os Shaders, Compute Units e filas de comandos de hardware residem[15].
- **sdma_v5_0_ip_block**: System Direct Memory Access, um motor DMA offload usado para cópias assíncronas de memória de alta velocidade sem sobrecarregar os command processors gráficos[32].
- **dce_virtual_ip_block / DCN 2.01**: Bloco de display responsável pelo varrimento de memória para o ecrã[32].

O processo iterativo da função amdgpu_device_ip_init executa duas fases metodologicamente separadas, essenciais para a resiliência do sistema:

1. **A fase sw_init (Software Initialization)**: O driver aloca todas as estruturas de dados necessárias na memória do sistema (RAM host), prepara os anéis de comandos (ring buffers) associados a cada bloco IP, dimensiona os contextos de fila e configura os ponteiros. Fundamentalmente, esta fase não comunica com o hardware[29].
2. **A fase hw_init (Hardware Initialization)**: O driver atinge fisicamente os registos MMIO da placa PCIe. É nesta fase que o microcódigo vital é carregado para a Static RAM (SRAM) da GPU, os registos são configurados para os seus estados padrão testados (conhecidos no código como "Golden Settings", instanciados na função gfx_v10_0_init_golden_registers), as interrupções de hardware são ativadas e o bloco IP é formalmente retirado do seu estado de suspensão ou reset[10].

Durante a análise dos relatórios de erros na comunidade Linux envolvendo a placa BC-250 e o carregamento do amdgpu, é frequente a observação de pânicos descritos como "hw_init of IP block failed -22" (o erro -22, tipicamente EINVAL, significa argumento inválido e indica falhas na decodificação do header de firmware ou recusa do hardware) ou failed -110 (ETIMEDOUT, significando que a GPU não respondeu após o microcódigo ser injetado)[35].

Na arquitetura WDDM que se pretende estruturar sem APIs gráficas completas, estas lógicas deverão estar centralizadas e contidas no processo de criação e inicialização do adaptador físico via interface DXGK (notavelmente no manuseamento interno invocado pelo KMD no momento da iniciação do dispositivo PnP). A replicação exata do sw_init e do hw_init em funções internas do driver Windows garantirá que a GPU não seja ativada sem as garantias estruturais necessárias, isolando potenciais falhas de timeout.

Análise Profunda da Gestão e Carregamento de Microcódigo (Firmware)

A peça crítica que viabilizou o suporte funcional e estável da Cyan Skillfish e da sua derivação direta na árvore de desenvolvimento do kernel foi o tratamento meticuloso do microcódigo (ucode). As GPUs modernas da AMD operam sob a égide de múltiplos microcontroladores embebidos que exigem a injeção inicial de blobs binários na memória do dispositivo.

O Commit "fix cyan_skillfish2 gpu info fw handling" e a sua Relevância

A investigação exigida aponta diretamente para a análise da submissão: Patch "drm/amdgpu: fix cyan_skillfish2 gpu info fw handling"[29]. Nas arquiteturas avançadas, como a RDNA 2, existe um ficheiro binário peculiar carregado antecipadamente, designado por cyan_skillfish_gpu_info.bin. Ao contrário do microcódigo destinado ao SMC (System Management Controller) ou ao CP (Command Processor), este binário não é um código executável, mas um cabeçalho de dados altamente estruturado[29].

Este firmware contém metadados vitais relativos à topologia física do silício. Informa o driver sobre as tabelas de tensão/frequência, limites acústicos e de consumo, e, de forma crítica para a placa BC-250, detalha o processo de "harvesting". O termo harvesting refere-se ao processo de fabrico em que CUs defeituosas ou não desejadas são desativadas a laser no die (neste caso, a APU inteira tem 36 CUs, mas a BC-250 opera apenas com 24)[1].

No processo sw_init original do AMDGPU, o driver realiza uma chamada a amdgpu_ucode_request para aceder a este ficheiro através da infraestrutura de firmware do kernel Linux (/lib/firmware/amdgpu/)[29]. Antes da correção aplicada por este patch, o código tentava processar as métricas de informação da GPU (especificamente os limites da Virtual RAM e as contagens de CUs) assumindo que o binário seguia a mesma estrutura rigorosa da variante cyan_skillfish original. A iteração cyan_skillfish2 causava anomalias na leitura, resultando numa alocação incorreta de anéis GFX ou num pânico durante a parametrização do GART[29]. A modificação na lógica de "fw handling" garantiu a validação rigorosa dos parâmetros nulos e adaptou os offsets de leitura dentro da estrutura do binário[29].

No âmbito do desenvolvimento do driver Windows (KMD), esta exigência dita que o ficheiro binário de informações e todos os microcódigos subsequentes terão de ser lidos diretamente do sistema de ficheiros do Windows, carregados do registo do sistema, ou incrustados de forma monolítica no ficheiro .sys do driver. A extração atempada dos metadados de harvesting determinará o número correto de Dispatchers a alocar, prevenindo colisões fatais durante a submissão computacional.

Direct Loading vs. PSP Loading: A Abordagem de Injeção de Firmware

Outra idiossincrasia arquitetural fundamental do Cyan Skillfish (especialmente a variante identificada pelas macros como AMD_APU_IS_CYAN_SKILLFISH2) é o mecanismo de carregamento de firmware[10].

Em chips Navi regulares (RDNA 1 e RDNA 2 padrão), o Platform Security Processor (PSP), frequentemente sob as iterações psp_v11_0 ou superiores, gere todo o arranque seguro (Secure Boot) dos blocos lógicos da GPU[33]. Sob o paradigma PSP, a CPU anfitriã submete os blobs binários para a RAM do sistema e sinaliza o coprocessador criptográfico embebido da AMD para os copiar para a memória interna, validá-los através de assinaturas criptográficas rigorosas RSA/SHA e, finalmente, injetá-los no SMC e no CP.

Contudo, observa-se que, para o Cyan Skillfish, a AMD e os seus engenheiros (nomeadamente através do commit de Lang Yu drm/amdgpu: use direct loading by default for cyan_skillfish2) introduziram um desvio comportamental deliberado no subsistema amdgpu_ucode.c através da função de triagem amdgpu_ucode_get_load_type[44].

Para o APU sinalizado como CYAN_SKILLFISH2, o código instrui o driver a preterir o PSP e invocar o método AMDGPU_FW_LOAD_DIRECT (vulgarmente conhecido como direct loading

ou front door loading)[44]. Nesta metodologia de carregamento direto, o driver kernel anfitrião assume a responsabilidade total. O driver acede aos registos MMIO da GPU e deposita sequencialmente e manualmente cada byte de firmware nas SRAMs internas dos processadores de comando.

A razão para esta anomalia está enraizada nas origens do silício. Sendo o BC-250 uma placa baseada no silício da consola da Sony, o processador de segurança (PSP v11.0.8) contém restrições e cadeias de confiança (TrustZone) proprietárias que rejeitariam qualquer firmware não assinado criptograficamente com as chaves específicas do ecossistema original[3]. Forçar o AMDGPU_FW_LOAD_DIRECT permite que o kernel anfitrião ignore a triagem do processador de segurança, carregando o microcódigo operativo elementar necessário para despertar o motor gráfico.

Para o projeto WDDM bare-metal focado unicamente no acesso total à GPU, implementar a rotina de Direct Load eliminará, de forma decisiva, uma imensa camada de complexidade associada à comunicação com o microcontrolador PSP e aos seus ring buffers encriptados. O KMD a desenvolver deve focar-se na escrita manual de bytes nas aberturas designadas do UVD/VCN e GFX.

Configuração da Memória, GMC v10 e a Virtualização da GPU (GPUVA)

A gestão da memória do Cyan Skillfish no Linux baseia-se fortemente no bloco IP Graphics Memory Controller versão 10 (gmc_v10_0)[19]. Como a BC-250 opera com uma topologia de Unidade de Processamento Acelerado (APU), recorre a uma Arquitetura de Memória Unificada (UMA). A memória GDDR6, totalizando 16 GB e providenciando uma largura de banda substancial de aproximadamente 448 GB/s, é partilhada nativa e diretamente entre os seis núcleos Zen 2 e os 24 CUs da GPU[1].

O Espaço de Endereçamento, ZRAM e Conflitos de UEFI

A gestão desta partilha no Linux dita que a UEFI/BIOS original da placa pode ser adaptada (frequentemente com recurso a ROMs modificadas pela comunidade) para estabelecer uma divisão rigorosa ou dinâmica. Os perfis incluem configurações de 512MB de pré-alocação dinâmica até segmentações rígidas, como 6 GB estritamente para a GPU e 10 GB para a CPU[1].

Esta dualidade de memória apresenta desafios significativos de estabilidade. O uso de pré-alocação dinâmica é sobejamente problemático em sistemas baseados em Linux que empregam ZRAM ou swaps agressivos em memória RAM, devido a colisões nos limites da memória partilhada entre as exigências do driver de vídeo e o gestor de memória virtual do kernel anfitrião[1]. Quando se desenhar o driver em WDDM, será imprescindível reportar rigidamente a totalidade da VRAM através de respostas exatas aos inquéritos DXGK_QUERYADAPTERINFOTYPE, concretamente os parâmetros de DXGKQAITYPE_PHYSICALADAPTERCAPS e o estado das memórias de segmento em DXGKQAITYPE_SEGMENTMEMORYSTATE[4]. Falhar a devida circunscrição das páginas de GDDR6 levará a corrupção imediata na transição para o ring buffer.

O GART e a Tradução Pela Interface WDDM

O bloco GMC gere o mapeamento das Translation Lookaside Buffers (TLBs) e das Tabelas de Página (Page Tables) da GPU[10]. Durante a subida do driver Linux AMDGPU, a função gmc_v10_0_sw_init constrói o esqueleto do Graphics Address Remapping Table (GART), seguida por gmc_v10_0_gart_enable que empurra a base do diretório de páginas para os registos do Host Data Path (mmHDP_HOST_PATH_CNTL)[10].
A passagem para um driver Windows impõe a substituição total do ecossistema GART do Linux pelo sistema de Paging Buffer do WDDM 2.0 (e arquiteturas superiores como o WDDM 3.x). O WDDM modernizou a gestão de GPU Memory Management Unit (GPU MMU) com suporte extenso a GPUVA (GPU Virtual Addressing).
Neste vetor arquitetural, o programador do Kernel Mode Driver (KMD) terá de:

1. **Definir Capacidades Virtuais:** O driver deve retornar suporte inequívoco ativando a flag GpuMmuSupported em conformidade com o Microsoft Compute Driver Model[4]. De forma complementar, como o Cyan Skillfish unifica a memória de alto desempenho, o suporte a IoMmuSupported pode mitigar ataques ou corrupções derivadas da comunicação DMA isolada via hardware (Shared Virtual Memory - SVM)[4].
2. **Implementar DDIs de Gestão de Tabelas:** O Dxgkrnl comandará a topologia MMU enviando chamadas para as funções DxgkDdiGetRootPageTableSize (para inquirir o alinhamento necessário dos diretórios RDNA 2) e DxgkDdiSetRootPageTable (para comandar o driver a instanciar a base da hierarquia GPUVA)[4].
3. **Mapeamento Host-Aperture:** Se for implementada a abertura do CPU (CPU Host Aperture) para transferências rápidas em System RAM, dever-se-ão incluir DxgkDdiMapCpuHostAperture e a contraparte DxgkDdiUnmapCpuHostAperture[4]. Sem estas chamadas, as cópias de dados estruturados e de Ring Buffers a partir do Ring0 do kernel falharão devido a falhas de tradução de endereços físicos na interface PCI. A função fornecida pela Microsoft, DxgkCbMapMemory, traduzirá os endereços físicos da memória de recursos atribuídos e viabilizará escritas puras MMIO[51].

A Arquitetura de Registos SoC15 e Acesso MMIO (Memory-Mapped I/O)

O ponto onde a construção de um driver bare-metal a partir do zero consumirá mais tempo de engenharia é na manipulação do barramento de comunicação física PCIe em si. O Linux AMDGPU abstrai toda essa pletora de comandos com macros de leitura e escrita fortemente enraizadas na estrutura base: RREG32_SOC15 (Read Register 32-bit System on Chip v15) e WREG32_SOC15 (Write Register 32-bit SoC15)[16].
A conceção destas macros foi impulsionada pela adoção transversal da filosofia IP Block em que cada subsistema (por exemplo, GC para Graphics/Compute, HDP para Host Data Path, SMUIO para o System Management Unit, DCE para a ponte de ecrã) opera quase como um pequeno microcontrolador separado[16].
O funcionamento da macro WREG32_SOC15(ip, inst, reg, value) opera na seguinte cascata:

1. **Seleção de Domínio:** Determina qual o bloco IP interpelado.
2. **Seleção de Instância:** Lê a instância desse bloco. Na topologia da BC-250 (e APUs equivalentes), existe geralmente apenas uma instância singular de cada bloco principal

(Instância 0), salvo exceções de multiplexagem em topologias unificadas[53].

3. **Cálculo de Offset Dinâmico:** Acopla a distância relativa do bloco IP ao endereço mestre PCIe.
4. **Resolução de Registo:** Adiciona a localização exata do registo, como por exemplo, a diretiva para ativar uma fila no bloco Graphics: mmCP_HQD_ACTIVE[53].
5. **Transação de Barramento:** Invoca um mecanismo de barramento (writel no Linux) para escrever o valor, forçando o hardware correspondente a responder.

Em ambientes de execução estritos como WDDM, e particularmente num contexto MCDM isento de gráficos, o driver base que o utilizador desenvolverá terá de replicar ou gerar uma gigantesca matriz de offsets e uma árvore lógica semelhante. A Microsoft providencia primitivas subjacentes via as port drivers como DxgkCbReadDeviceSpace, mas o KMD deve conter a lógica para navegar a matriz[52]. A complexidade acrescida da Cyan Skillfish impõe novamente a referida necessidade de não adotar offsets lidos passivamente no início da bota, mas basear-se no array interno testado que a AMD introduziu na função de substituição cyan_skillfish_reg_base_init, protegendo as instâncias de endereços que não respeitam a disposição tradicional da Navi 10/14 ou Navi 21/22[10].

Submissão de Comandos: Command Processors e Ring Buffers (O Coração da GPU)

Com o firmware injetado, os relógios SMU estabilizados e o GART traduzindo memória, o cerne operacional na comunicação bidirecional sem o encapsulamento de APIs assenta na criação e gestão das Hardware Queues e Ring Buffers. Na placa RDNA 2, esse processo requer orquestração extrema de submissões assíncronas via acesso à memória principal[15].

A Arquitetura Circular e a Implementação KIQ

A GPU e o driver comunicam primordialmente não por interrupções contínuas ou envios de MMIO byte-a-byte, mas através da submissão para uma fila circular em memória (Ring Buffer). O Host (a CPU e o driver KMD) coloca sequencialmente instruções binárias nessa fila. O Host mantém e envia um ponteiro que aponta para o último comando escrito (Write Pointer - WPTR). Em contrapartida, a GPU (Command Processor), funcionando de forma assíncrona, verifica esse ponteiro, ingere os comandos pendentes, executa o seu rasto e atualiza internamente o seu próprio registo de progresso (Read Pointer - RPTR).

Contudo, na arquitetura avançada GFX10.x, a iniciação do fluxo de comandos não começa nos anéis convencionais de Shaders GFX ou anéis puros de Compute. Em vez disso, a fundação começa com o **Kernel Interface Queue (KIQ)**[53].

A KIQ é uma fila de submissão altíssima e privilegiada destinada em absoluto ao controlo do espaço de kernel. O seu propósito primordial é instanciar, modificar ou destruir outras instâncias secundárias de filas do hardware[53]. Todo e qualquer mapeamento subsequente das filas reais de Computação (Compute Queues) tem de fluir através de um pacote submetido primeiro ao KIQ através das diretivas PACKET3_MAP_QUEUES[56].

As Funções Críticas: amdgpu_ring_write e amdgpu_ring_commit

No ecossistema do kernel aberto da AMD, toda a codificação das submissões passa pela

utilização iterativa da função amdgpu_ring_write, seguida por uma invocação terminal à função amdgpu_ring_commit[55].
As operações injetadas através da função formam estruturas PM4 (Protocol Message 4). A variante de pacote principal nestes processadores chama-se **PACKET3**. Um fluxo lógico num RDNA 2 para limpar a cache l1/l2 ou sinalizar o estado do hardware implica frequentemente:

1. A invocação de amdgpu_ring_write(ring, PACKET3(PACKET3_SET_RESOURCES, 6)). Este comando dita um pacote tipo 3 destinado a fixar variáveis de estado virtuais[55].
2. Nas subsequentes submissões amdgpu_ring_write (que devem totalizar exatas seis DWORDS estipuladas no cabeçalho), enviam-se a máscara de Virtual Memory ID (vmid_mask), a especificação do anel (KIQ ou outro) e os delimitadores superior e inferior dos endereços GPU de 64 bits a atuar[55].
3. O ato solene desta fase é encerrado por amdgpu_ring_commit(ring).

O "commit" é vital[53]. Esta ação no Linux emite o que é intrinsecamente denominado como "tocar na campainha" (Doorbell Ring). O amdgpu_ring_commit agarra no novo endereço absoluto do ponteiro Write (WPTR), insere-o no registo mapeado em hardware PCIe designado DOORBELL específico para essa fila. Esta inversão no registo MMIO notifica o microcontrolador do Command Processor RDNA que novo trabalho de empacotamento foi depositado em system memory. O loop que se lhe segue no kernel costuma realizar um udelay ou espelhar a confirmação com um retorno a 0xDEADBEEF numa zona segura (scratch) de teste para assegurar que os comandos não colapsaram a máquina, validando que a submissão passou[55].

Transposição Paradigmática para o WDDM e o MCDM Windows

Para o projeto ambicioso que o utilizador idealizou — desenhar do zero uma estrutura base (KMD) que dispense APIs gráficas tradicionais para uma gestão unicamente bare-metal, computacional ou de teste na placa Cyan Skillfish —, impõe-se a adoção estrita das restrições WDDM 2.0 (e subsequentes) ou, preferencialmente, do Microsoft Compute Driver Model (MCDM)[4].
No MCDM, as funções base não requerem renderização tradicional (omitindo complexos shaders D3D num primeiro momento) mas exigem estritamente controlo inexpugnável das filas DMA virtuais.
O KMD transcreverá as etapas do AMDGPU da seguinte forma arquitetural obrigatória[4]:

1. **DxgkDdiCreateProcess**: O Dxgkrnl do Windows invocará este callback quando um utilizador instanciar um recurso na GPU. O KMD a desenvolver para a BC-250 deverá aproveitar este ponto para criar uma estrutura de dados de processamento associada a um VMID (de 0 a 15), isolando a memória correspondente das outras execuções[4].
2. **DxgkDdiCreateContext**: Equivalente a declarar ou solicitar a instanciação formal de um Ring (seja via envio de pacote PACKET3_MAP_QUEUES no anel KIQ base) para poder processar comandos[50].
3. **DxgkDdiSubmitCommandVirtual**: O nó vital. Quando um software arbitrário invocar um envio computacional, o Kernel Windows chamará este DDI entregando um DMA buffer na memória host endereçada virtualmente. O KMD desenvolvido assumirá este DMA

buffer[4]. Neste ponto, caberá ao driver traduzir, higienizar (se necessário) os cabeçalhos das submissões (agindo exatamente como o amdgpu_ring_write), encadear o pacote na fila física circular da placa BC-250 e submeter um Doorbell trigger no MMIO correto para informar o hardware da atualização do WPTR[4].

O Ecossistema de Ecrã: Display Core Next (DCN) 2.01

Embora a visão primária para uma base de acesso GPU total se foque no computacional e processamento assíncrono bare-metal, todo e qualquer driver que almeje escalar para WDDM de vídeo completo deverá suportar e endereçar o subsistema de saída de monitor. Na perspetiva da placa Cyan Skillfish (BC-250), o processador encarregue dessa função é o idiossincrático **DCN 2.01**[3].

A evolução e suporte desta máquina beneficiaram das manobras de engenharia pioneiras de Andy Nguyen (na tentativa bem-sucedida de implementar Linux na estrutura congénere da PlayStation 5). Esta iniciativa obrigou à escrita explícita e fundição de suporte na árvore mestre do kernel amdgpu de ficheiros unicamente dedicados às métricas desta implementação de ecrã altamente singular. Zhan Liu, um engenheiro da AMD, formalizou a disponibilização desse bloco nas patches integradas no diretório drivers/gpu/drm/amd/display/dc/dcn201/[34].

O PC tradicional e as APUs contemporâneas basearam-se fortemente no DCN 2.0 (Navi 10 original) ou no DCN 2.1 (APUs mobile Renoir). O DCN 2.01 atua como uma ilha retroativamente compatível para hardware proprietário[34]. O driver contém módulos expressamente talhados para as árvores de frequências do ecrã e multiplexadores lógicos, invocando dcn201_clk_mgr_construct e dcn201_resource_construct[34]. Estas lógicas lidam com o fornecimento das tabelas de temporização da porta singular DisplayPort 1.4 existente na placa (a qual sofre de peculiaridades frequentes com áudio e taxas de refrescamento caso seja sujeita a adaptadores passivos HDMI)[1].

Sob o modelo MCDM (Microsoft Compute Driver Model) que dispensa especificamente o encargo imediato do ecossistema de DDI Display e Renderização pesada, a engenharia do bloco DCN 2.01 pode ser inicialmente adiada. Se o projeto WDDM quiser operar na sua glória PnP completa com sinal de imagem no ecrã boot, será premente codificar a implementação nativa das DDIs críticas como DxgkDdiSystemDisplayEnable, associando o Surface de frame buffer linear primário aos controladores CRTC, Hubbubs e pontes digitais formatadas pelo DCN 2.01 do Cyan Skillfish[51]. Adicionalmente, importa notar que o BC-250 não suporta qualquer bloco físico ativo de Video Core Next (VCN), obstando imediatamente qualquer tentativa de construir rotinas de codificação/descodificação AV1/HEVC via hardware a nível de driver; todos os fluxos baseiam-se em decodificação software no CPU anfitrião[1].

Suporte na Camada de Driver Gráfico User-Space: O Projeto Mesa

Com o Kernel Mode Driver (KMD) perfeitamente delineado, é impossível atingir a aceleração gráfica e computacional pura sem a vertente do User Mode Driver (UMD), uma camada de espaço de utilizador que formula as listas DMA a enviar para as DDIs do kernel. Na contraparte baseada em Linux (que espelhará futuramente o UMD DXGI/Vulkan sob o ambiente de

Windows), o projeto Mesa tem encabeçado uma integração formidável para a vertente GFX1013 do RDNA 2 nas APUs Oberon[3].
As submissões do suporte Cyan Skillfish na camada de biblioteca de gráficos Mesa (notoriamente nas ramificações 25.3.x e 26.1-devel) não envolveram meramente adicionar novos PCI IDs à listagem genérica. Remendos fulcrais como o de Anna Maniscalco amd/addrlib: Add more GFX1013 GPUs demonstram a reestruturação da *Address Library* (addrlib) — o cerne algorítmico do driver encarregue de computar alinhamentos herméticos, mapeamentos, transições e texturas tiling específicas (swizzling) exigidas por este die específico na gestão GART[3].
Outros patches Mesa resolveram problemas idiossincráticos a nível da rasterização fundamental e das falhas no Z-buffer, como a imposição de um workaround específico (implementado pelo flag condicional para contornar falhas do comando *clear value* do recurso *HTILE TC Z*) estritamente para o renderizador de superfícies do asic GFX1013[14]. Estes desenvolvimentos na API em Vulkan provam que a implementação estrutural da placa BC-250 acarreta desvios suficientes na arquitetura das RDNA convencionais, alertando para a imperatividade de que, após um KMD bare-metal base funcional, a transição para renderização nativa de polígonos ou primitivos computacionais envolverá imensas otimizações de cálculo de endereços específicos a esta unidade.

Síntese de Mapeamento Arquitetural (WDDM para KMD BC-250)

Para consolidar as vastas referências num projeto coeso, a estruturação base sem API gráfica traduz o fluxo do kernel aberto para o ambiente Microsoft através de mapeamentos diretos de chamadas e funções lógicas, garantindo um percurso sólido na conceção do WDDM.

Lógica Inicial no AMDGPU (Linux)	**Componente Estrutural Requisitado (Windows WDDM/MCDM)**	**Implementação Recomendada no Driver Novo para a BC-250**
amdgpu_pci_probe e macro amdgpu_device_init	DxgkDdiAddDevice acoplado ao sequencial DxgkDdiStartDevice	Inicia a inspeção aos BARs nativos (Base Address Registers). Interceta obrigatoriamente os IDs 13FE ou 143F relativos à Cyan Skillfish, aloca buffers internos e estabelece de imediato a base local de offsets MMIO.[4]
Carregamento amdgpu_ucode_request	Rotinas Internas Autoritárias (Injeção de FW)	Devido aos constrangimentos de

(via PSP vs. Direct Load)		TrustZone remanescentes, processar obrigatoriamente o binário local e injetar via carregamento frontal de MMIO (Direct Loading) evitando submissões inválidas ao processador de segurança.[29]
gmc_v10_0_sw_init e gestão GART	DxgkDdiGetRootPageTableSize / DxgkDdiSetRootPageTable	Instrui imperativamente o Dxgkrnl Windows de que a placa requer alinhamento virtual (GpuMmuSupported = TRUE), declarando as tabelas e o volume de VRAM rigidamente fixado para escapar a colisões do sistema de permuta dinâmico ZRAM/Paging da UEFI host.[1]
Acessos base via RREG32_SOC15 / WREG32_SOC15	Mapeamento virtual MMIO (DxgkCbReadDeviceSpace)	O KMD terá que instanciar o desdobramento e tradução dinâmica entre a instância lógica do SoC15 RDNA 2 e os endereços virtuais baseados nos offsets da topologia da ASRock / AMD[16].
Ciclo contínuo de amdgpu_ring_write e amdgpu_ring_commit	DxgkDdiSubmitCommandVirtual	A artéria principal do driver WDDM a desenvolver. Capta um DMA Buffer em memória virtual do utilizador, empacota com metadados e PM4 (PACKET3) para processamento da fila de comandos (ex: KIQ) e acorda o Hardware através do toque na campainha (Doorbell).[50]

Bloco dce_virtual_ip_block (KMS)	DxgkDdiSystemDisplayEnable ou isenção MCDM	Se o driver permanecer puramente computacional "bare-metal", focar apenas nos DDI MCDM (ignorando o display temporariamente). Se for gráfico, mapear diretamente sobre os multiplexadores singulares do módulo de ecrã dcn201.[4]

A fundação do WDDM que se objetiva desenvolver para a BC-250 tem de abraçar estas diretivas não convencionais. Assegurar a implementação do AMDGPU_FW_LOAD_DIRECT é o passo fulcral que retirará a placa do estado inerte, abrindo as portas do ring buffer de comandos e das filas KIQ, almejando em última análise a maturação de um driver capaz de acesso absoluto à poderosa malha computacional e memória unificada do Cyan Skillfish, e pavimentando a ascensão para futuras implementações totais (com suporte a API).

Referências citadas

1. Specifications - AMD BC250 Documentation, https://elektricm.github.io/amd-bc250-docs/hardware/specifications/
2. Introduction to the AMD BC-250, https://elektricm.github.io/amd-bc250-docs/getting-started/introduction/
3. PS5 Linux hack leads to first Mesa update for Sony GPU - VideoCardz.com, https://videocardz.com/newz/ps5-linux-hack-leads-to-first-mesa-update-for-sony-gpu
4. mcdm-implementation-guidelines.md - windows-driver-docs - GitHub, https://github.com/MicrosoftDocs/windows-driver-docs/blob/staging/windows-driver-docs-pr/display/mcdm-implementation-guidelines.md
5. MCDM Kernel-Mode Driver Implementation Guidelines - Windows drivers | Microsoft Learn, https://learn.microsoft.com/en-us/windows-hardware/drivers/display/mcdm-implementation-guidelines
6. Documentação completa para placas AMD BC250 - Jogos no Linux em hardware de ex-mineração (Versão reduzida da APU do PS5) : r/linux4noobs - Reddit, https://www.reddit.com/r/linux4noobs/comments/1p3yvbr/comprehensive_documentation_for_amd_bc250_boards/?tl=pt-br
7. AMD Cyan Skillfish [BC-250] - Linux Hardware Database, https://linux-hardware.org/?id=pci:1002-13fe-1022-0000
8. amdgpu_drv.c source code [linux/drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c] - Codebrowser, https://codebrowser.dev/linux/linux/drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c.

html
9. File: amdgpu_drv.c - Debian Sources, https://sources.debian.org/src/linux/6.1.8-1/drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c/
10. Diff - b14ffae378aa1db993e62b01392e70d1e585fb23^1..b14ffae378aa1db993e62b01392e70d1e585fb23 - kernel/common - Git at Google, https://android.googlesource.com/kernel/common/+/b14ffae378aa1db993e62b01392e70d1e585fb23%5E1..b14ffae378aa1db993e62b01392e70d1e585fb23/
11. Diff - 385f6c065c0f4ab25ebb04d81f93bb0abcd3fbda^1..385f6c065c0f4ab25ebb04d81f93bb0abcd3fbda - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/385f6c065c0f4ab25ebb04d81f93bb0abcd3fbda%5E1..385f6c065c0f4ab25ebb04d81f93bb0abcd3fbda/
12. Mesa 26.1.0 Release Notes / 2026-05-06, https://docs.mesa3d.org/relnotes/26.1.0.html
13. Mining Board Meets Bazzite: The $300 DIY Steam Machine | Retro Handhelds, https://retrohandhelds.gg/mining-board-meets-bazzite-the-300-diy-steam-machine/
14. Mesa 25.3.0 Release Notes / 2025-11-14, https://docs.mesa3d.org/relnotes/25.3.0.html
15. Diff - a594533df0f6ca391da003f43d53b336a2d23ffa^1..a594533df0f6ca391da003f43d53b336a2d23ffa - kernel/common - Git at Google, https://android.googlesource.com/kernel/common/+/a594533df0f6ca391da003f43d53b336a2d23ffa%5E1..a594533df0f6ca391da003f43d53b336a2d23ffa/
16. Diff - 68a00586559f5fce3bb163f4618f1b71ef05b4e7^1..68a00586559f5fce3bb163f4618f1b71ef05b4e7 - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/68a00586559f5fce3bb163f4618f1b71ef05b4e7%5E1..68a00586559f5fce3bb163f4618f1b71ef05b4e7/
17. Diff - 4992eb41ab1b5d08479193afdc94f9678e5ded13^2..4992eb41ab1b5d08479193afdc94f9678e5ded13 - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/4992eb41ab1b5d08479193afdc94f9678e5ded13%5E2..4992eb41ab1b5d08479193afdc94f9678e5ded13/
18. Diff - 574b311f3f8f5395e33ab0e242e0820c84ee15aa^2..574b311f3f8f5395e33ab0e242e0820c84ee15aa - kernel/msm.git - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/msm.git/+/574b311f3f8f5395e33ab0e242e0820c84ee15aa%5E2..574b311f3f8f5395e33ab0e242e0820c84ee15aa/
19. drivers/gpu/drm/amd/amdgpu/gmc_v10_0.c · master · OpenSource / copier-linux-5.15.131, https://ipads.se.sjtu.edu.cn:1312/opensource/copier-linux-5.15.131/-/blob/master/drivers/gpu/drm/amd/amdgpu/gmc_v10_0.c?ref_type=heads

20. Changelog in Linux kernel 5.10.237 - OpenNET, https://www.opennet.ru/kernel/5.10.237.html
21. Vulnerabilidades - INCIBE, https://www.incibe.es/index.php/incibe-cert/alerta-temprana/vulnerabilidades?field_vul_product=&page=2
22. Test GPU (AMD Radeon RX 6700 XT) · Issue #222 · geerlingguy/raspberry-pi-pcie-devices, https://github.com/geerlingguy/raspberry-pi-pcie-devices/issues/222?timeline_page=1
23. [SOLVED] No Output From 7900XTX GPU via OVMF PCI Passthrough (Again) (Page 2) / Kernel & Hardware / Arch Linux Forums, https://bbs.archlinux.org/viewtopic.php?id=288756&p=2
24. [PATCH V13 00/51][FINAL] Color Pipeline API w/ VKMS - public-inbox listing - Freedesktop.org, https://lore.freedesktop.org/wayland-devel/20251115000237.3561250-4-alex.hung@amd.com/T/
25. Diff - fc86e5c9909b1b9a80ff0e7946279dfae6b96ded^2..fc86e5c9909b1b9a80ff0e7946279dfae6b96ded - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/fc86e5c9909b1b9a80ff0e7946279dfae6b96ded%5E2..fc86e5c9909b1b9a80ff0e7946279dfae6b96ded/
26. Navi Reset Bug Kernel Patch v2 : r/VFIO - Reddit, https://www.reddit.com/r/VFIO/comments/jjy74y/navi_reset_bug_kernel_patch_v2/
27. Security Bulletin: IBM Security Guardium is affected by multiple kernel vulnerabilities, https://www.ibm.com/support/pages/security-bulletin-ibm-security-guardium-affected-multiple-kernel-vulnerabilities-1
28. linux/drivers/gpu/drm/amd/amdgpu/amdgpu_discovery.c at master - GitHub, https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/amdgpu_discovery.c
29. amdgpu_device.c source code [linux/drivers/gpu/drm/amd/amdgpu/amdgpu_device.c] - Codebrowser, https://codebrowser.dev/linux/linux/drivers/gpu/drm/amd/amdgpu/amdgpu_device.c.html
30. amd « drm « gpu « drivers - ~agd5f/linux - freedesktop.org git repository browser, https://cgit.freedesktop.org/~agd5f/linux/tree/drivers/gpu/drm/amd/amdgpu/amdgpu_device.c?h=amd-staging-drm-next&id=b989531b1f192a77c739a2976953e241d78229a3
31. [RFC v2 12/15] drm/amdgpu: introduce IP block iterators to reduce duplicated code, https://lists.freedesktop.org/archives/amd-gfx/2025-January/118684.html
32. [PATCH 06/29] drm/amdgpu: set ip blocks for cyan_skillfish - Freedesktop.org, https://lists.freedesktop.org/archives/amd-gfx/2021-July/066808.html
33. [PATCH 00/29] Cyan Skillfish support - Freedesktop.org,

https://lists.freedesktop.org/archives/amd-gfx/2021-July/066803.html
34. [PATCH 02/02 v2] drm/amd/display: add cyan_skillfish display support - Freedesktop.org, https://lists.freedesktop.org/archives/amd-gfx/2021-September/069486.html
35. 108781 – 4.19 Regression - Hawaii (R9 390) boot failure - Invalid PCC GPIO / invalid powerlevel state / Fatal error during GPU init, https://bugs.freedesktop.org/show_bug.cgi?id=108781
36. ROCm 2.0 cannot load hawaii firmware · Issue #640 - GitHub, https://github.com/ROCm/ROCm/issues/640
37. Amdgpu: Fatal error during GPU init on 5.19.8 (works on 5.19.6) - Fedora Discussion, https://discussion.fedoraproject.org/t/amdgpu-fatal-error-during-gpu-init-on-5-19-8-works-on-5-19-6/75827
38. AMD iGPU passthrough attempt - General - Qubes OS Forum, https://forum.qubes-os.org/t/amd-igpu-passthrough-attempt/6766
39. Linux 6.18 - LKML: Linus Torvalds, https://lkml.org/lkml/2025/11/30/341
40. [PATCH 6.6 00/93] 6.6.119-rc1 review - Lkft-triage - lists.linaro.org, https://lists-ec2.96boards.org/archives/list/lkft-triage@lists.linaro.org/thread/O7FNWHWR5B7MVACQZY33K6Y7WNQO2D35/
41. [PATCH 6.1 000/568] 6.1.159-rc1 review - Lkft-triage - lists.linaro.org, https://lists-ec2.96boards.org/archives/list/lkft-triage@lists.linaro.org/thread/ITJWL5UB7TFNEO3RI7DCIWYVF34KNGF6/?sort=thread
42. SUSE SLES15: cluster-md-kmp-64kb / cluster-md-kmp-default / dl... | Tenable®, https://www.tenable.com/plugins/nessus/296680
43. Linux 6.17.11 - LWN.net, https://lwn.net/Articles/1049556/
44. [PATCH 21/29] drm/amdgpu: use direct loading by default for cyan_skillfish2, https://lists.freedesktop.org/archives/amd-gfx/2021-July/066822.html
45. [pull] amdgpu, amdkfd drm-next-5.16 - Freedesktop.org, https://lists.freedesktop.org/archives/dri-devel/2021-October/326742.html
46. Quick Reference - AMD BC250 Documentation - GitHub Pages, https://elektricm.github.io/amd-bc250-docs/reference/quick-reference/
47. git.odroid Code Review - drivers/gpu/drm/amd/amdgpu/gmc_v9_0.c, https://git.odroid.com/gitweb?p=linux.git;a=blob;f=drivers/gpu/drm/amd/amdgpu/gmc_v9_0.c;h=72f8018fa2a836572b9c898785bb99deecc1ca91;hb=a3d9103ebfa03824d255060fc2c11ac94e3ef441
48. Riktlinjer för mcdm-Kernel-Mode-drivrutinsimplementering - Windows drivers, https://learn.microsoft.com/sv-se/windows-hardware/drivers/display/mcdm-implementation-guidelines
49. MCDM Kernel-Mode ドライバー実装ガイドライン - Windows drivers | Microsoft Learn, https://learn.microsoft.com/ja-jp/windows-hardware/drivers/display/mcdm-implementation-guidelines
50. D3Dkmddi.h header - Windows drivers - Microsoft Learn, https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/d3dkmddi/
51. Display/Graphics overview - Windows drivers | Microsoft Learn,

https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/_display/
52. Dispmprt.h header - Windows drivers - Microsoft Learn, https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/dispmprt/
53. Diff - cce3b573a52a41dd7face9dbf745f10f9bf4632b^2..cce3b573a52a41dd7face9dbf745f10f9bf4632b - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/cce3b573a52a41dd7face9dbf745f10f9bf4632b%5E2..cce3b573a52a41dd7face9dbf745f10f9bf4632b/
54. [PATCH openEuler-5.10 000/119] Backport 5.10.83 LTS - Kernel, https://mailweb.openeuler.org/hyperkitty/list/kernel@openeuler.org/thread/3TPWG54ACKDWWH5VMMHYK4TGUSLPWY2N/
55. linux/drivers/gpu/drm/amd/amdgpu/gfx_v11_0.c at master - GitHub, https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/gfx_v11_0.c
56. linux/drivers/gpu/drm/amd/amdgpu/amdgpu_amdkfd_gfx_v9.c at master - GitHub, https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/amdgpu_amdkfd_gfx_v9.c
57. git.odroid Code Review - linux.git/commitdiff, https://git.odroid.com/gitweb?p=linux.git;a=commitdiff;h=937eea297e26effac6809a0bf8c20e6ca9d90b9a;hp=126a34061eec2df05a5a28052edefd4e6125f31c
58. Diff - 15d90b242290f228166ea79ee1cc2db6b31a2143^2..15d90b242290f228166ea79ee1cc2db6b31a2143 - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/15d90b242290f228166ea79ee1cc2db6b31a2143%5E2..15d90b242290f228166ea79ee1cc2db6b31a2143/
59. Merge branch 'drm-next-4.12' of git://people.freedesktop.org/~agd5f/linux into drm-next - ~airlied/linux - Official DRM kernel tree, https://cgit.freedesktop.org/~airlied/linux/commit/?id=8cd3ac52963f2e99f4c21d1c9ce89531ce66c2d6
60. [PATCH 21/29] drm/amdgpu: add init support for GFX11 (v2) - Freedesktop.org, https://lists.freedesktop.org/archives/amd-gfx/2022-April/078513.html
61. Merge tag 'drm-next-5.4-2019-08-09' of git://people.freedesktop.org/~agd5f/linux into drm-next - drm/drm - DRM kernel graphics driver development tree - freedesktop.org git repository browser, https://cgit.freedesktop.org/drm/drm/commit/?id=e7f7287bf5f746d29f3607178851246a005dd398
62. Visão geral de exibição/elementos gráficos - Windows drivers - Microsoft Learn, https://learn.microsoft.com/pt-br/windows-hardware/drivers/ddi/_display/
63. [PATCH 00/02] cyan skillfish display support - Freedesktop.org, https://lists.freedesktop.org/archives/amd-gfx/2021-September/069481.html
64. [PATCH 02/02] drm/amd/display: add cyan_skillfish display support - Freedesktop.org, https://lists.freedesktop.org/archives/amd-gfx/2021-September/069483.html
65. Información general sobre visualización y gráficos - Windows drivers - Microsoft

Learn, https://learn.microsoft.com/es-es/windows-hardware/drivers/ddi/_display/